

PHP MVC Migration - Complete Student Submission

Topic: Converting a Plain PHP CRUD Application to MVC Architecture

Concrete Implementation Domain: Library Book Borrowing System

1. Assignment Overview

This submission applies the PHP MVC migration guide to a concrete CRUD project: a Library Book Borrowing System. The goal is to refactor a plain PHP CRUD application into a clean Model-View-Controller architecture without using any framework.

The solution includes a written migration analysis, MVC folder structure, component responsibility tables, route/action mapping, security checklist, diagrams, XAMPP running steps, and a complete PHP MVC code skeleton.

2. Part 1 - Understand the Existing Plain PHP CRUD Application

Before migration, a typical plain PHP CRUD application usually mixes database connection, SQL queries, form handling, and HTML output in one or several loosely organized files. The migration process begins by identifying these mixed responsibilities.

Component in old code	Typical content	MVC destination
Database connection	new PDO(...) block at the top of a PHP file	config/database.php
Business logic / SQL	SELECT, INSERT, UPDATE, DELETE queries	models/BookModel.php
Form handling	\$_POST and \$_GET processing	controllers/BookController.php
HTML output	tables, forms, echo statements, mixed PHP/HTML	views/books/*.php

3. Part 2 - Planned MVC Folder Structure

The final project uses a single public entry point. The browser accesses public/index.php, while models, controllers, views, and config files are loaded internally by PHP.

```
php_mvc_library_crud/
├── config/
│   └── database.php
├── models/
│   └── BookModel.php
├── controllers/
│   └── BookController.php
├── views/
│   ├── partials/
│   │   ├── header.php
│   │   └── footer.php
│   └── books/
│       ├── index.php
│       ├── show.php
│       ├── create.php
│       └── edit.php
```

```

├── public/
│   ├── index.php
│   ├── assets/
│   │   └── style.css
├── database.sql
└── README.md

```

4. Part 3 - Step-by-Step Migration Plan

Step 1 - Extract the Database Connection

Move the PDO connection into config/database.php and create one reusable getConnection(): PDO function. This prevents duplicated database connection code.

Step 2 - Create the Model

Create models/BookModel.php. This file contains only database operations such as getAll(), getById(), create(), update(), and delete(). It must not echo HTML or read \$_POST directly.

Step 3 - Create the Controller

Create controllers/BookController.php. This file reads HTTP requests, validates input, calls BookModel methods, and loads the correct view.

Step 4 - Create the Views

Create views/books/index.php, show.php, create.php, and edit.php. Views contain only HTML and small PHP display logic. SQL queries must not be placed in views.

Step 5 - Create the Front Controller

Create public/index.php. This file reads the action parameter and calls the corresponding controller method.

Step 6 - Add Security Practices

Use prepared statements for SQL, cast IDs to integers, use htmlspecialchars() in views, validate required fields, and redirect after POST actions.

5. MVC Responsibility Table

Layer	File / Class	Responsibility
Model	models/BookModel.php	Handles database queries and returns book data as arrays.
View	views/books/index.php	Displays all books in an HTML table.
View	views/books/create.php	Displays the add book form.
View	views/books/edit.php	Displays the edit book form.
View	views/books/show.php	Displays one book detail page.
Controller	controllers/BookController.php	Handles requests, validates input, calls the Model, and loads Views.
Config	config/database.php	Creates a reusable PDO connection.
Front Controller	public/index.php	Reads action from URL and routes to controller methods.

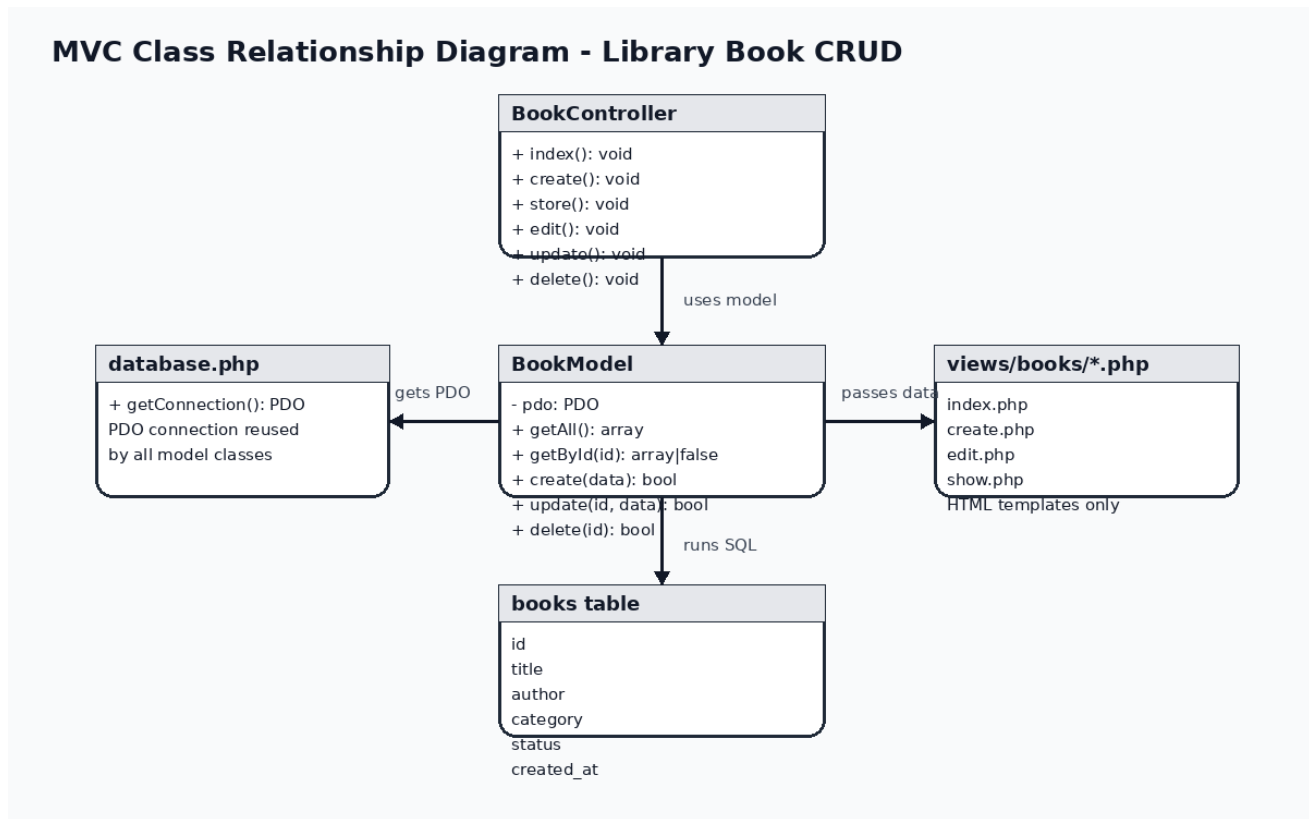
6. Route / Action Table

HTTP	URL	Controller@Action	Purpose
GET	/public/index.php?action=index	BookController@index	List all books.
GET	/public/index.php?action=show&id=1	BookController@show	View details of one book.
GET	/public/index.php?action=create	BookController@create	Show add book form.

POST	/public/index.php?action=store	BookController@store	Save a new book.
GET	/public/index.php?action=edit&id=1	BookController@edit	Show edit form.
POST	/public/index.php?action=update	BookController@update	Update an existing book.
GET	/public/index.php?action=delete&id=1	BookController@delete	Delete a book and redirect.

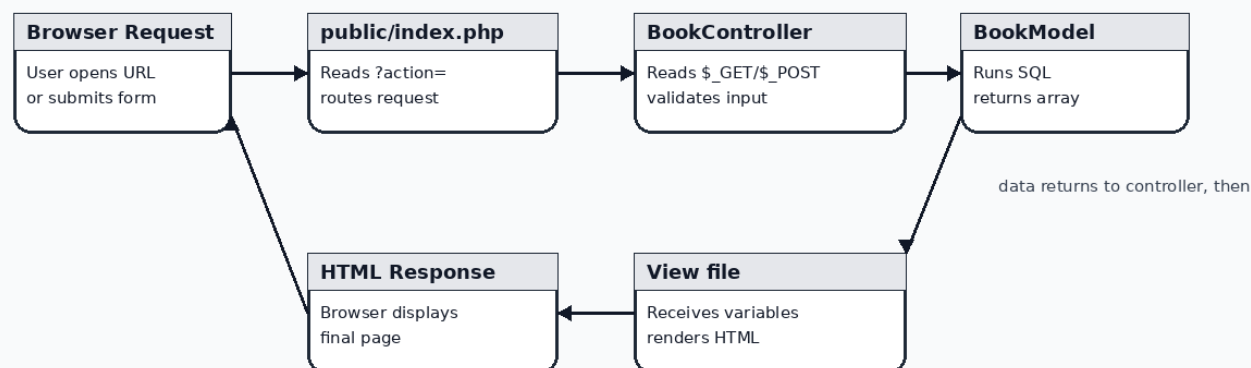
7. UML and Data Flow Diagrams

The UML diagram below shows how the main MVC classes and files relate to one another.



The data flow diagram below shows how a browser request moves through public/index.php, the Controller, the Model, the View, and finally back to the browser as an HTML response.

MVC Data Flow Summary



8. Migration Mapping from Plain PHP to MVC

Old plain PHP part	Problem	New MVC location
PDO connection copied in many files	Hard to maintain and risky when credentials change	config/database.php
SELECT query inside list page	View is mixed with database logic	BookModel::getAll()
INSERT query inside form page	Form page handles too many responsibilities	BookController::store() + BookModel::create()
UPDATE query inside edit page	HTML and SQL are mixed	BookController::update() + BookModel::update()
DELETE logic in link target file	No central routing	BookController::delete()
HTML table generated inside SQL file	Model outputs presentation	views/books/index.php

9. Security and Quality Checklist

- All SQL statements that receive user input use prepared statements with placeholders.
- All values printed in Views use htmlspecialchars() to reduce XSS risk.
- All IDs from URL or form input are cast to integers before being used.
- After create, update, and delete actions, the controller redirects to avoid duplicate form submissions.
- Database credentials are kept only in config/database.php.
- Views do not contain SQL queries.
- Models do not echo HTML.
- Controllers do not contain raw SQL queries.

10. Running Steps in VS Code and XAMPP

1. **Copy folder:** Copy php_mvc_library_crud to C:\xampp\htdocs\php_mvc_library_crud.
2. **Start services:** Open XAMPP Control Panel and start Apache and MySQL.
3. **Create database:** Open phpMyAdmin and import database.sql. This creates mvc_library_db and the books table.
4. **Open project in VS Code:** Use File > Open Folder and select C:\xampp\htdocs\php_mvc_library_crud.
5. **Run in browser:** Visit http://localhost/php_mvc_library_crud/public/index.php.

6. **6. Test CRUD:** Add a book, view book details, edit a book, and delete a book.

11. Reflection Questions

1. Which files are Model, View, and Controller?

The Model is BookModel.php because it communicates with the database and returns data. The Views are the files inside views/books because they display HTML only. The Controller is BookController.php because it receives user actions, calls the Model, validates input, and loads the correct View.

2. Why should SQL not be placed inside Views?

SQL should not be placed inside Views because Views are responsible only for presentation. If SQL is mixed with HTML, the application becomes difficult to maintain and the MVC boundary is broken.

3. Why should prepared statements be used?

Prepared statements reduce SQL injection risk because user input is passed as parameters instead of being directly concatenated into SQL strings.

4. Why redirect after POST actions?

Redirecting after create, update, and delete actions prevents duplicate form submissions when the user refreshes the page. It also returns the user to a stable GET page.

5. What can be improved next?

The next improvement is adding authentication for librarian/admin users, adding a BorrowModel for borrowing history, and using autoloading to load classes automatically.

12. Submission Package

Recommended submission folder:

```
PHP_MVC_Migration_Library_Submission/
|
|— 01_Report/
|   |— PHP_MVC_Migration_Library_Complete_Submission.docx
|   |— PHP_MVC_Migration_Library_Complete_Submission.pdf
|
|— 02_Source_Code/
|   |— php_mvc_library_crud.zip
|
|— README_Submission.txt
```

If the submission system allows only one file, submit the PDF report. If it allows multiple files, submit both the PDF report and the source code ZIP.

13. Key Code Summary

This section summarizes the main code files included in the ZIP package.

File	Main responsibility
config/database.php	Defines getConnection(): PDO.
models/BookModel.php	Contains getAll(), getById(), create(), update(), delete().
controllers/BookController.php	Handles index, show, create, store, edit, update, and delete actions.
views/books/*.php	Displays book list, detail, create form, and edit form.

public/index.php	Routes ?action= to the correct controller method.
database.sql	Creates the database, table, and sample data.